Low-cost LiDAR spatial reasoning: answering a question versus computing its answer

Nassim Arifette
MVA — Nuages de Points et Modélisation 3D (NPM3D)

October 21, 2026

Abstract

LiDAR-LLM [1] couples a frozen LiDAR encoder to a large language model so that spatial questions about a driving scene can be answered in natural language. Our original project reimplemented a reduced version of that architecture and showed, honestly, that it could not be trained on the hardware available. This report changes the question rather than the hardware. Holding perception fixed, we ask whether a spatial question about a LiDAR scene is better answered by *producing* the answer or by *computing* it, and we ask it of one model so that the two are genuinely comparable: the same frozen Qwen2.5-1.5B-Instruct either answers from the object list, or writes a small program that a deterministic executor runs on that same list. On 200 held-out questions from three unseen nuScenes drives the executed mode reaches 0.685 against 0.505; paired question by question it is alone correct on 67 and alone wrong on 31 ($p = 3.5 \times 10^{-4}$), and its counting error is 0.68 against 2.16 at equal numeric coverage. The sharper instrument is a scene-dependence control: of thirteen questions asked verbatim of two scenes with different answers, the executed mode is right on all thirteen and the direct mode on one. Two findings about the data matter beyond this comparison. The six nuScenes-QA relations are recovered empirically from box geometry — uniform 60° bearing sectors fit the reference answers better than five alternatives including the six camera frusta — and 23% of nuScenes annotations carry *zero* LiDAR returns while nuScenes-QA counts them anyway, so filtering objects by LiDAR support, the obvious thing to do, costs 49 accuracy points. Nothing here is trained, and the whole study runs on one free T4.

1 Introduction

Interpreting a LiDAR scene in natural language is attractive for autonomous driving: a system that can answer "how many cars are ahead of us?" exposes its scene understanding in a form a human can audit. LiDAR-LLM [1] pursues this by aligning a LiDAR representation with the embedding space of a language model through a three-stage curriculum, with GPT-4 [8] improving the captions that form the training data and LLaMA-7B as the model that learns from LiDAR.

Our first report reimplemented a reduced version of this architecture. Section 2 restates what that implementation did and, more usefully, what it failed to establish. Section 3 argues for a different question, one the same hardware can answer.

The shift is motivated by an observation about counting and directional questions. When the correct answer to "how many cars are in front?" is determined by the object list and a geometric predicate, asking a generative model to emit the digit is asking it to re-derive, inside its weights, a computation that is already exact and essentially free. Whether that re-derivation buys anything is testable, and this report tests it on one model used two ways.

2 The original re-implementation and what it did not show

The first version implemented the LiDAR-LLM pipeline in reduced form: a small PointNet encoder, a view-aware transformer with six view-position embeddings, an MLP adapter projecting into the token embedding space, and GPT-2 [7] fine-tuned with LoRA [9]. Data came from nuScenes-mini [2] with nuScenes-QA [3] questions: 323 training samples from 8 scenes, 81 validation samples from 2 scenes.

It produced a debugged pipeline and no evidence. Training ran on laptops with roughly 6 GB of RAM and no GPU. The functional check was that the model could be driven to emit a fixed string regardless of the LiDAR input, which shows that gradients flow and nothing more.

Three weaknesses shape what follows.

The evaluation could not separate competence from priors. nuScenes-QA answers are skewed towards "no" and small integers. With no question-only control, an accuracy figure cannot distinguish a model that reads the scene from one that has learned the answer distribution. Real-3DQA [12] shows how severe this is: on SQA3D a model fine-tuned on question–answer pairs *with no 3D input at all* matches or beats existing 3D-LLMs, and filtering the blind-answerable questions collapses LEO's exact match from 49.4% to 14.3%.

The split invited leakage. Splitting questions rather than scenes puts different questions about the same LiDAR sample on both sides, rewarding "this scene has three cars".

The target task did not need a language model. Counting, existence and directional questions have answers that are functions of the object list.

3 A question the available hardware can answer

We fix perception and vary only the reasoning stage. Every system receives exactly the same object list, and the question becomes:

> *At a fixed language model and fixed scene data, is it more reliable to produce the answer to a spatial question, or to produce an executable program and let a deterministic executor compute the answer?*

Posed between *two uses of one model*, this is answerable without confounds. A learned answer head and a program-writing model differ in architecture, in training and in what they see, so a gap between them says little; the same checkpoint, the same decoding and the same objects differ in exactly one respect.

Relation to existing work. The idea is not ours. CLEVR [4] established the template: represent a question as a functional program over a small catalogue of operations — filter, unique, relate, count, exist, query, compare — and execute it on the scene graph. Neural-symbolic VQA [5] then showed that recovering such a program and executing it on a structured scene representation is far more accurate on CLEVR than predicting the answer end to end. Our qualitative result is a reproduction of that finding on real driving data with a frozen 1.5B model and no training at all; the contribution is the controlled measurement and the two data findings of Section 4, not the idea.

Closer in domain, ZSVG3D [10] produces programs from language for zero-shot 3D grounding. KLDrive [11] merges multi-source evidence into a scene knowledge graph and lets a frozen Qwen3-7B agent reason over it through a Plan–Execute–Observe loop, reaching 65.04% overall and 64.46% on counting against 18.45% for its strongest baseline. Retrieve-to-Localize [13] makes

the complementary argument for coordinates. BEV-LLM [14] extends LiDAR-LLM with camera images through BEVFusion. NuScenes-SpatialQA [15] finds a spatially enhanced VLM improving on qualitative questions while remaining uncompetitive on quantitative ones — precisely the counting gap targeted here. 3DGraphLLM [16] learns a scene-graph representation carrying semantic relations.

Two things prevent a direct numerical comparison. These systems change more than the reasoning module — KLDrive's accuracy is obtained with three detectors over 39,704 selected questions under a random 8:2 split that is not stratified by scene, so part of the gain is perception and part is protocol. And "frozen LLM" does not mean cheap: KLDrive runs on RTX A6000s, Retrieve-to-Localize trains on four A6000s, BEV-LLM's 1B variant takes 48 hours on eight A100s. The experiment below runs in under three minutes on one free T4.

4 Method

4.1 Object-level scene representation

A scene is a list of objects, each with a canonical class (the ten nuScenes detection classes), a centre, a box size, a yaw, a detection score and, where the annotation carries one, a *status* — moving, parked, stopped, standing, sitting/lying down, with rider, without rider. All coordinates are in the ego frame: x forward, y left, z up. nuScenes boxes and released detection results are in global coordinates, so the conversion through the sample's ego pose and the quaternion-to-yaw step are performed explicitly and in one place; a sign error there mirrors the scene and would be indistinguishable from a reasoning failure.

An unknown status is treated as neither the attribute nor its negation. A traffic cone carries no attribute, so it is not "standing" and not "not standing"; counting it under the negation would assert something the annotation does not support.

4.2 Directions as bearing sectors, recovered rather than assumed

nuScenes-QA uses six spatial relations. Our first implementation used eight 45° octants, which silently answered a different question: an object at a bearing of 50° is "front left" under one convention and "left" under the other, and "left" does not exist in the six-relation vocabulary at all.

Rather than replace one assumption with another, the convention was *measured* against the benchmark's own answers. Over the 68 ego-anchored directional questions of the evaluation manifest we scored five candidate definitions by exact agreement with the reference answer:

Candidate convention	Exact	Count MAE	Bias
Uniform 60° sectors, y left, no range limit	**76.5%**	1.24	−1.14
Same, with y pointing right	72.1%	0.76	−0.76
The six nuScenes camera frusta	67.6%	1.76	−0.62
Eight 45° octants	61.8%	1.62	−1.62
Overlapping quadrants ($x > 0$, $y > 0$, …)	51.5%	3.86	+2.62

Table 1: Recovering the relation definition from the gold answers. The camera row uses the real calibration: each camera's optical axis and horizontal field of view are computed from `calibrated_sensor.json` and the intrinsics, and an object belongs to a camera if it projects inside that image.

The six-camera hypothesis is tempting — six relations, six cameras, named `CAM_FRONT_LEFT` and so on — and it is wrong: the frusta overlap and are unequal in width (64–89° from the intrinsics), and they fit worse than uniform sectors. Every maximum-range cut from 25 to 60 m also makes the fit worse, so nuScenes-QA counts objects at any distance.

The residual is structured: on the development split the executor agrees with 50/50 non-directional questions, 17/18 axial ones (front, back) and 24/32 diagonal ones, with *front left* alone at 5/10. The boundaries were deliberately *not* fitted to close that gap. An executor whose sector edges were tuned on the reference answers would help the branch that computes with those edges far more than the branch that only reads them in prose, and the comparison of Section 6 would be measuring the tuning.

Objects closer than 1 m to the ego origin are excluded from directional queries, because a 10 cm localisation error flips the sign of their bearing. Relative directions ("left of the truck") are ambiguous between the viewer's frame and the object's own frame; both are implemented, and the viewer-centric reading is the default, matching nuScenes-QA.

4.3 What the LiDAR returns say about the labels

nuScenes annotates every object a human could see, and records how many LiDAR points fall inside each box. Building a scene from LiDAR, the natural move is to keep only annotations the sensor actually hit. On the mini split that discards 4,278 **of** 18,538 **annotations** — 23% — **for having zero LiDAR returns**.

It is also wrong for this benchmark. Holding the programs fixed and changing only the object list, over the 125 counting questions that both conditions answer numerically:

Object list	Accuracy	Under-counting	MAE
Keep only annotations with ≥ 1 LiDAR return	0.336	1.672	1.76
Keep every annotation	**0.824**	**0.568**	**0.69**

Table 2: Same questions, same programs, same geometry; only the object list changes. Filtering by LiDAR support costs 49 accuracy points.

So nuScenes-QA's reference answers count objects the LiDAR never saw. This is a fact about the benchmark rather than about reasoning, and it is worth stating plainly because the filter is the obvious engineering choice and it is expensive. Every experiment below therefore uses the unfiltered list, and the choice is recorded in the manifest so both compared systems inherit it.

4.4 From question to program

A program is an operation (COUNT, EXISTS, COMPARE) over a selector: a category or category group, an optional status, an optional region (a bearing sector, near, far, or a radius), and an optional anchor making the region relative to another object. Two parsers emit this language. A deterministic rule grammar serves as a reference and as a baseline; the frozen language model is the system under test.

The grammar declines what it cannot represent rather than guessing. This matters more than it sounds: an earlier version had no slot for a status word and silently dropped it, so "what number of stopped things are there" was executed as "how many objects are there" and answered 71 against a reference of 10. Making the grammar able to carry the attribute and the preamble referent raised its end-to-end accuracy on all 5,916 mini questions from 0.156 (drop what you cannot express) to 0.231 at the same coverage, with accuracy-when-answering rising from 0.632 to 0.856.

4.5 Execution

Executing a program is a filter followed by an arithmetic operation. It is deterministic, has no learned parameters, and returns the objects it selected, which is what allows an error to be attributed to perception rather than to reasoning. A definite reference whose resolution is ambiguous ("the truck", when the scene has three) is refused rather than resolved by taking the

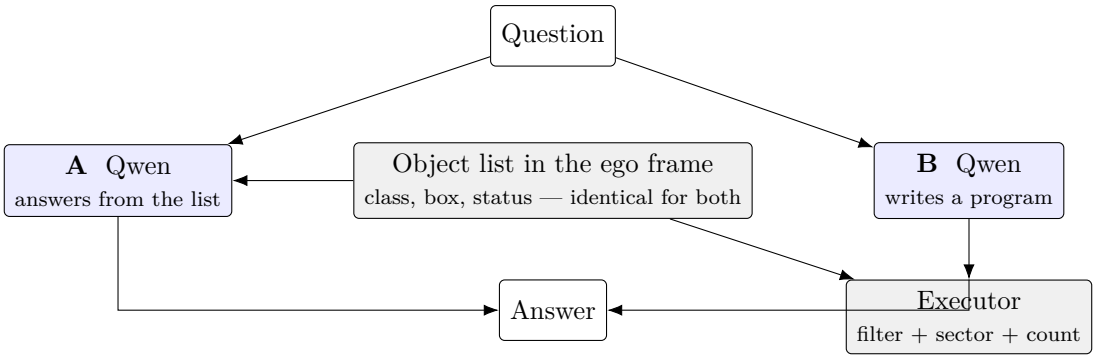


Figure 1: The two modes. Grey is fixed and shared; blue is the same frozen checkpoint used two ways. Both branches see the same objects and are told the same conventions.

first match. CLEVR avoids such questions at generation time, calling them ill-posed [4]; on data we did not generate, well-posedness has to be detected instead, and a refusal is counted as a wrong answer throughout.

5 Experimental protocol

5.1 The frozen example set

Both systems are measured on one set, fixed before either ran (`manifests/first_comparison.json`): 100 development and 200 evaluation questions drawn from nuScenes-QA mini, over nuScenes ground-truth annotations.

Split by scene, never by sample. Consecutive samples of one drive show the same vehicles, so splitting by sample would make the evaluation set a paraphrase of the development set. Development holds seven drives, evaluation three, with no overlap.

Selection by what the question asks, not by what a parser manages. Families are assigned from the question's surface structure — does it name an attribute, a direction, a reference object — using the lexicons directly. Keeping the questions the grammar happens to parse would quietly select the examples that favour the executed branch. Of the 5,916 mini questions 1,626 are in scope; the 4,290 excluded are recorded with their reason (2,199 belong to a template outside this scope, 1,005 bind an attribute to another object, 86 name no known class).

What fixes the scope, precisely. It is the *rule grammar*, not the flat language. The flat language has operations for identifying an object, querying its attribute and comparing two objects' attributes, so it could attempt the `object`, `status` and `comparison` templates. The rule grammar cannot, and the rule grammar is what produces the reference programs that let the executor be checked before any model runs (Section 5.2). Restricting the first comparison to what can be independently verified is the reason; it is a smaller claim than "what the language can express", and Section 8 measures the difference.

Four families , in increasing structural demand: category only ("are there any cars"), category with an attribute ("how many parked cars"), category in an ego-relative region, and category anchored on another object ("behind the truck"). The reference answer is available to the evaluator and not to either inference function.

An oracle perception condition. Object lists are ground-truth annotations, not detections. This is deliberate: it fixes perception so the comparison is about what each system does with a scene rather than who sees it better. It also bounds the claim — nothing in Section 6 speaks to detection quality.

5.2 The executor, checked before any model ran

Attributing a wrong answer to the model requires knowing the executor is right. On the development split, reference programs produced by the rule grammar and read by hand were executed on the frozen object lists with no model involved: 91/100, with the category-only and attribute families at 25/25. All nine disagreements were read and categorised: *eight* are the diagonal-sector residual of Section 4.2, and *one* is a question whose definite reference ("the not standing pedestrian") matches 22 objects — nuScenes-QA answers it anyway, so its one-hop templates appear to read the reference existentially rather than as a definite description. That disagreement is left as a refusal and charged against the executed system, not excused.

5.3 Matched conditions

Both modes use the same checkpoint (Qwen2.5-1.5B-Instruct) loaded through one shared runtime, the same dtype, greedy decoding, and the same 256-token budget. Both receive the same object rows at the same rounding, in an order that does not depend on the question — no list pre-filtered by what is being asked, no precomputed counts. Both are told the same conventions: the ego frame, the six sectors with their numeric bounds, and the treatment of unknown status.

Two safeguards are worth naming. Prompts longer than the context budget are *refused* rather than trimmed, because trimming deletes objects and the loss would be scored as a reasoning error; and every generation records whether it stopped on its own or ran out of budget, after an earlier run lost 26% of its programs to a 96-token budget, 97% of them to truncation — a configuration choice that had been read as a model failure. In the runs reported here no generation was truncated.

The caches are separate and keyed differently. A program depends on the question; a direct answer depends on the question *and* the scene. Sharing a key would serve one scene's answer for another's, destroying the control of Section 6.2.

6 Results: one model, two ways of using it

Nothing was changed between the two splits. The development result was measured twice, before and after a fix to cost accounting and batching, and every accuracy figure was identical.

The aggregate hides the measurement that matters. Paired question by question:

	A only	B only	both	neither	sign test
development	15	**38**	36	11	$p = 2.2 \times 10^{-3}$
evaluation	31	**67**	70	32	$p = 3.5 \times 10^{-4}$

The advantage survives the move to unseen scenes, and the counting improvement is not an artefact of abstention: the two modes produce an integer on practically the same questions (1.000 against 0.988).

The sign test is optimistic, and here is by how much. It assumes independent questions. Two hundred questions over three drives are not independent — answers within a drive share a scene — so the effective sample is nearer three than two hundred. Per drive the margin is +0.212, +0.188 and +0.138 (B at 0.758, 0.725, 0.569 against A at 0.545, 0.536, 0.431): same sign, same order of magnitude, on all three. That consistency is the honest claim. A cluster bootstrap

	development (n=100)		evaluation (n=200)	
	A, direct	B, executed	A, direct	B, executed
Accuracy (all)	0.510	**0.740**	0.505	**0.685**
Answers	1.000	0.860	1.000	0.910
Accuracy when answering	0.510	0.860	0.505	0.753
Numeric coverage	1.000	0.946	1.000	0.988
Under-counting	2.08	0.54	1.84	0.34
Over-counting	0.35	0.00	0.31	0.34
Mean absolute error	2.43	**0.54**	2.16	**0.68**
Input / output tokens	124 311 / 200	112 095 / 5 756	217 272 / 400	178 402 / 9 731
Generation time (T4)	47.3 s	59.3 s	58.8 s	101.0 s

Table 3: Both modes of Qwen2.5-1.5B-Instruct. Refusals and invalid programs count as wrong answers in the accuracy row. The evaluation split shares no scene with development.

over three clusters can only draw ten distinct resamples and returns a spuriously tight interval; we do not report one.

	development		evaluation	
Family	A	B	A	B
Attribute ("how many parked cars")	0.640	**1.000**	0.547	**0.962**
Category only ("are there any cars")	0.760	0.720	0.680	0.840
Anchored on another object	0.320	0.440	0.352	0.444
Ego-relative region	0.320	0.800	0.442	0.465

Table 4: Per family. Only the first row is stable across the two splits.

Only one family generalises cleanly. The attribute family is a large, consistent win (+0.36 then +0.42): applying "parked" as a filter is exactly what an executor does and a 1.5B model does not. The other three move substantially between the splits — the ego-relative gain falls from +0.48 to +0.02, and the category-only comparison changes sign. At 25 to 54 questions per family these are too noisy to carry an argument, and we do not build one on them. Had only the development split been run, the ego-relative row would have looked like the headline finding; it is the clearest argument in this report for holding scenes back.

The direct model does not scan the list. Its errors are not marginal. Where the reference is 7, 9 or 10 it answers 1, 3 and 2; on preamble questions it answers 1 almost regardless of the scene. With forty-odd object rows in the prompt, a 1.5B model emits a small number rather than enumerating. Its under-counting (1.84–2.08) dwarfs its over-counting (0.31–0.35), the signature of not looking rather than of looking badly. Its answer *format* was never the obstacle: all three hundred direct outputs parsed.

Cost. The two modes are closer than the accuracy gap suggests. On evaluation A consumes 217,272 input and 400 output tokens, B 178,402 input and 9,731 output: B reads less, because it never sees the objects, and writes twenty-four times more, because it writes a program. Wall-clock generation is 59 s against 101 s on one T4, both batched identically.

6.1 Does the gap close with a larger model?

The result above leaves two explanations entangled. The direct mode may be losing because a 1.5B model cannot write the program it would need internally, or simply because it cannot count rows in a forty-line table of floats — a weakness a larger model might not share. Repeating the comparison at two more sizes, changing nothing else, separates them.

		accuracy		counting MAE			
Model	precision	A	B	A	B	B answers	invalid
Qwen2.5-1.5B	fp16	0.505	0.685	2.16	0.68	0.910	8
Qwen2.5-7B	fp16	0.505	0.825	1.84	0.44	0.985	1
Qwen2.5-32B	int4	0.580	**0.845**	1.65	0.60	0.995	**0**

Table 5: The same 200 held-out questions, the same three drives, the same prompts and decoding. "Invalid" counts programs the model emitted that are not programs. The 32B point is quantised to 4 bits because the checkpoint is 65 GB in bfloat16 and the card holds 40; quantisation can only hurt it, so that row understates the large model.

Scale does not rescue the direct mode. It gains 0.075 across a twenty-one-fold increase in parameters, and 0 of that between 1.5B and 7B. The identical 0.505 at the first two sizes is not a cached result: the 7B changes its answer on 89 of the 200 questions and flips correctness on 52 of them, 26 in each direction. It answers differently and is wrong just as often. Whatever the direct mode is failing at, it is not something 4.7 times the parameters fixes.

What scale does fix is program validity. Programs the executor cannot parse fall from 8 to 1 to 0, and the answered fraction rises from 0.910 to 0.995. The executed mode gains 0.160 from 1.5B to 7B and then only 0.020 more at 32B: the remaining errors are no longer about writing a well-formed program.

The gap widens before it stabilises — +0.180, +0.320, +0.265 — and the paired tables stay decisive at every size ($p = 3.5 \times 10^{-4}$, 1.2×10^{-11}, 1.1×10^{-9}). The honest reading is not that language models cannot do this. It is that, over the range we could afford to test, the capacity gained by scaling went into writing better programs and almost none of it into answering from the table directly.

6.2 The scene-dependence control

Thirteen evaluation questions appear verbatim on two scenes with different reference answers. A system that does not read the objects gives the same answer to both and scores on whichever scene matches its prior.

	groups	A, direct		B, executed	
		one answer to all	right on every scene	one answer to all	right on every scene
development	5	2	0	1	4
evaluation	13	4	**1**	**0**	**13**

On the evaluation split the executed system never gives one answer to a whole group and is correct across all thirteen; the direct model is correct across a group once. No aggregate accuracy reveals that, and this is the instrument Real-3DQA [12] argues for.

7 Where the executed system fails

An accuracy of 0.685 says nothing about what to do next. The 31.5% that is wrong contains four problems with four different remedies, and they are separable without a model: where the model's program matches the deterministic grammar's and the answer is still wrong, the program is not at fault.

Failure	n	share	what it asks for
Program says something else	27	43%	better prompt, examples, bigger model
wrong operation	22		`count` written for an existence question
wrong or dropped status	2		`not with rider` for `without rider`
other	3		
Program right, answer wrong	18	29%	the executor and the data
Refused at execution	10	16%	ambiguous (8) or missing (2) reference
Malformed output	8	13%	a budget, or constrained decoding
Total	63	100%	

Table 6: The executed system's 63 evaluation failures, triaged.

Three conclusions follow, and they decide where effort should go.

The bottleneck is writing the program, not executing it. Forty-five of 63 failures (71%) are the model producing a program that does not match the question, or no program at all. Only 18 (29%) survive to the executor and disagree there, and those are the diagonal-sector residual of Section 4.2. Widening the target language — the obvious next engineering move, and one we deliberately did not make — would address almost none of this.

The largest single failure is a format mismatch, not a misunderstanding. In 22 cases the model writes `count` for a question asking `exists`: "are there any moving cars to the front left of me?" becomes a counting program. The selector is usually right; the answer is then an integer, which can never match a yes/no reference. One third of all failures is a model that understood the scene query and chose the wrong return type.

The malformed outputs have one cause. All eight emit `"category": "*"` for "things", a token outside the vocabulary, although the prompt lists the ten classes *and* states the rule mapping "thing" to `object`. The instructions are not at fault; a 1.5B model is. This is the clearest single thing to re-test with a larger model, and the one category of failure that grammar-constrained decoding would eliminate by construction.

8 Does a wider language help?

Everything above is measured inside one target language: an operation over one or two selectors, each carrying a category, an attribute, a region and at most one anchor. That shape covers the counting and existence templates and refuses the rest. The obvious next move — and the one the error analysis of Section 7 argues against — is to widen it.

We built the wider language anyway, and measured it, because "obvious next move" is not a result.

8.1 The second target language

It is CLEVR's representation [4], adapted to nuScenes: a program is a list of typed nodes, each naming a function, its constant argument and the indices of earlier nodes it consumes, in topo-

logical order. The catalogue has nineteen functions — `scene`, `filter_category`, `filter_status`, `unique`, `relate`, `relate_ego`, `and_`, `or_`, `same_status`, `count`, `exist`, `query_category`, `query_status`, `equal_value`, the integer comparisons — with input and output types checked before execution, so an ill-typed program is a reported error and never a plausible answer.

Three adaptations are claims about the data rather than conveniences. The attributes are nuScenes' ten classes and seven statuses, not CLEVR's size, colour, material and shape; there is no colour, which is why colour questions are unanswerable here and not merely unimplemented. There is an ego vehicle, so `relate` (anchored on a resolved object) and `relate_ego` (anchored on the sensor platform) are separate functions. And `unique` can fail: CLEVR prunes ill-posed questions when it generates them, but we did not generate ours, so well-posedness is detected at run time and reported.

The wider language admits a wider question set. A second manifest, built by the same rules on the same drives, covers nine families instead of four and 5,830 of the 5,916 mini questions — 98.5%, against 27.5% for the flat scope. The 86 it still excludes name no class the adapter knows.

8.2 Result

Qwen2.5-1.5B, wide scope ($n = 280$)	A, direct	B, flat	C, typed
Accuracy (all)	**0.521**	0.457	0.296
Answers	1.000	0.571	0.346
Accuracy when answering	0.521	0.800	**0.856**

Table 7: A wider language made things worse. Paired, the direct mode beats the typed one 102 to 39 ($p = 1.1 \times 10^{-7}$) and the flat one beats it 65 to 20 ($p = 1.0 \times 10^{-6}$).

The typed language is the most accurate of the three *when it produces a program at all*, and it produces one for barely a third of the questions. That is the whole of the result: expressiveness bought nothing because the model could not use it.

Where it fails is specific. Of 280 typed programs, 165 do not type-check. Seventy use a constant outside the vocabulary. Eighty-five wire the wrong type into a function, and *fifty-one of those eighty-five are a missing* `unique` — the model passes a set where `relate`, `query_status` or `same_status` requires one object.

The same difficulty, moved. On the same questions the flat language produces 240 valid programs and then loses 80 of them at execution: 42 ambiguous references and 38 missing ones. Both languages are defeated by the same thing — resolving a definite reference in a real scene — and they differ only in where it surfaces. The flat language performs uniqueness implicitly inside the executor and fails there; the typed language makes it an explicit node and fails at the type check. Making the step visible did not make the model take it.

Checked before claiming it. The obvious suspicion is that the typed language is merely stricter about spelling, since many of its vocabulary errors are plurals. It is not: the flat language rejects `"trailers"` exactly as the typed one does. The comparison is not an artefact of normalisation.

What this changes about the recommendation. The failure triage of Section 7 found that 71% of the flat language's errors happen before the executor. Widening the target language addresses none of those, and this experiment shows it adds a harder generation task on top. The language was not the binding constraint. That is a negative result, it cost one afternoon, and it

is the reason the typed catalogue ships in the repository without being wired into the headline comparison.

9 A preliminary study on synthetic scenes

Before the nuScenes protocol existed, the same executor was compared against *trained* baselines on a synthetic corpus, under the earlier 45° octant convention. Those numbers are reported here as a preliminary study, not as results on nuScenes-QA, and the convention alone makes them non-comparable with Section 6.

Scenes are sampled with nuScenes-like class frequencies, questions are generated *from* programs so every gold program is known, and a detector-failure model (12% drops, 5% duplicates, 0.6 m jitter) produces a "detected" list. **This simulates detector behaviour; it does not measure a detector.** No 3D detector was run anywhere in this project, and no claim about detection quality is made from it.

System	Accuracy	Count MAE	Pair acc.	Params
Majority answer	0.296	—	—	0
Question only (blind)	0.656 ± 0.012	0.722	0.000	35 863
Deep sets	0.655 ± 0.020	0.654	0.022	120 855
Deep sets + counter	0.644 ± 0.018	0.534	0.076	129 176
Symbolic	$\mathbf{0.916 \pm 0.027}$	**0.132**	**0.643**	**0**

Table 8: Synthetic corpus, 45° octants, means over seeds $0, 1, 2$. Pair accuracy requires both members of a contrast pair — one question text, two scenes with different answers — to be correct.

Neither learned model beats the question-only control. On contrast pairs the gap is starker: a blind model must score zero, deep sets reaches 0.022 and the counter 0.076, against 0.643. On the accuracy column one could argue the learned models use the scene a little; on contrast pairs they essentially do not.

Two cautions that the nuScenes work later confirmed. First, benchmark design changes the conclusion: on an unbalanced version of the same corpus, where 75% of answers mean "nothing there", the learned models appear to beat the blind baseline while their contrast-pair accuracy stays near zero. Second, these learned baselines were also run on real nuScenes-QA questions, where deep sets does *not* reliably beat the blind prior — paired by seed the margins are -0.021, $+0.032$, $+0.025$ — and where replacing ground-truth objects with simulated detections moves it by $+0.019$, -0.030, $+0.010$. A model whose score is unchanged when the 3D input is corrupted has not been shown to use it.

10 Discussion

What this establishes. At a fixed language model, fixed scene data and fixed conventions, emitting an executable program was substantially more reliable than emitting the answer: 0.685 against 0.505 on held-out drives, with a paired advantage of 67 to 31 and a counting error three times smaller at equal coverage. The scene-dependence control separates the two modes more sharply than any aggregate. The study is also cheap enough to repeat: nothing is trained, and the whole comparison is three minutes on a free T4.

What it does not establish. Branch B brings hand-written code that implements the benchmark's semantics, verified at 91/100. Its ceiling is high by construction, and the question is really whether the *model* can write the program — which it does validly 92–96% of the time,

reaching 0.753 when it does. The result is not evidence that language models cannot reason about space. The baseline is also weak in a specific way: a 1.5B model asked to count rows in a 44-line table of floats is being tested on something it is known to be poor at, and part of A's disadvantage may be context length rather than reasoning. Objects are ground truth, so nothing here speaks to perception. And the evaluation rests on three drives.

The one experiment that would settle the open question. Repeating the comparison with a substantially larger checkpoint, changing nothing else, separates the two explanations currently confounded: "the model cannot write the program" and "the model cannot count from a table". If the direct mode catches up at scale, the result is a fact about model size; if it does not, it is a fact about the division of labour. Section 7 predicts which failures should move — the 8 malformed outputs and the 22 wrong return types are exactly the kind a more capable model fixes.

Why the DSL was frozen. A composable, CLEVR-style catalogue of typed operations [4] — `filter`, `unique`, `relate`, `same_status`, `query`, with explicit input and output types — is implemented in the repository and deliberately *not* wired into any experiment. It would widen coverage from four families to the whole benchmark, but Table 6 says the current failures are overwhelmingly not expressiveness. Building a new language and changing model size at once would leave neither attributable.

Why the distillation line was dropped. The previous report proposed distilling the fine-tuned LLM into a smaller student. Distillation reduces the cost of a demonstrated capability; it cannot create one, and no spatial competence was established in the teacher.

11 Conclusion

The original project established that the LiDAR-LLM architecture can be wired up on a laptop, and nothing more. This revision asks a narrower question that the same hardware answers: given one frozen 1.5B model and one object list, is it better to produce the answer or to produce a program? On held-out nuScenes drives, the program. The margin is large, paired, and consistent across all three drives, and the scene-dependence control shows the direct mode largely reciting template answers while the executed mode tracks the scene.

The two findings most likely to be useful elsewhere are not about the comparison. The six nuScenes-QA relations are uniform 60° bearing sectors, recovered by fitting the reference answers rather than assumed, and the appealing six-camera reading is wrong. And 23% of nuScenes annotations have no LiDAR return at all while the benchmark counts them, so filtering objects by LiDAR support — the obvious thing to do when building from a point cloud — costs 49 accuracy points on otherwise correct programs.

References

[1] S. Yang et al. *LiDAR-LLM: Exploring the Potential of Large Language Models for 3D LiDAR Understanding*, 2023.

[2] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, O. Beijbom. *nuScenes: A Multimodal Dataset for Autonomous Driving.* CVPR, pp. 11618–11628, 2020.

[3] T. Qian, J. Chen, L. Zhuo, Y. Jiao, Y.-G. Jiang. *NuScenes-QA: A Multi-Modal Visual Question Answering Benchmark for Autonomous Driving Scenario.* AAAI, pp. 4542–4550, 2024.

[4] J. Johnson, B. Hariharan, L. van der Maaten, L. Fei-Fei, C. L. Zitnick, R. Girshick. *CLEVR: A Diagnostic Dataset for Compositional Language and Elementary Visual Reasoning*. CVPR, 2017. arXiv:1612.06890.

[5] K. Yi, J. Wu, C. Gan, A. Torralba, P. Kohli, J. B. Tenenbaum. *Neural-Symbolic VQA: Disentangling Reasoning from Vision and Language Understanding*. NeurIPS, 2018. arXiv:1810.02338.

[6] T. Yin, X. Zhou, P. Krähenbühl. *Center-Based 3D Object Detection and Tracking*. CVPR, pp. 11784–11793, 2021.

[7] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever. *Language Models are Unsupervised Multitask Learners*. OpenAI, 2019.

[8] OpenAI. *GPT-4 Technical Report*, 2023.

[9] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685, 2021.

[10] Z. Yuan, J. Ren, C.-M. Feng, H. Zhao, S. Cui, Z. Li. *Visual Programming for Zero-shot Open-Vocabulary 3D Visual Grounding*. CVPR, pp. 20623–20633, 2024. arXiv:2311.15383.

[11] Y. Tian, J. Zhang, Z. Wang, X. Ren, X. Yu, O. Gungor, T. Rosing. *KLDrive: Fine-Grained 3D Scene Reasoning for Autonomous Driving based on Knowledge Graph*. arXiv:2603.21029, March 2026.

[12] X. Ma et al. *Do 3D Large Language Models Really Understand 3D Spatial Relationships?* ICLR, 2026. arXiv:2603.23523.

[13] B. Park, G. Moon, J. Kim, S. Hwang. *Retrieve-to-Localize: Bridging Large Language Models and LiDAR Geometry for Spatial Grounding*. arXiv:2609.29835, September 2026.

[14] F. Brandstätter, E. Schütz, K. Winter, F. Flohr. *BEV-LLM: Leveraging Multimodal BEV Maps for Scene Captioning in Autonomous Driving*. IEEE Intelligent Vehicles Symposium (IV), pp. 345–350, 2025. arXiv:2507.19370.

[15] K. Tian, J. Mao, Y. Zhang, J. Jiang, Y. Zhou, Z. Tu. *NuScenes-SpatialQA: A Spatial Understanding and Reasoning Benchmark for Vision-Language Models in Autonomous Driving*. ICCV Workshops, pp. 4567–4576, 2025. arXiv:2504.03164.

[16] T. Zemskova, D. Yudin. *3DGraphLLM: Combining Semantic Graphs and Large Language Models for 3D Scene Understanding*. ICCV, 2025. arXiv:2412.18450.